



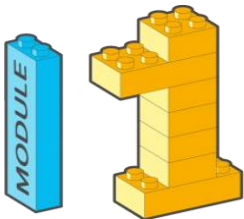
VIT[®]

Vellore Institute of Technology
(Deemed to be University under section 3 of UGC Act, 1956)

BCSE307L_PRINCIPLES OF COMPILER DESIGN



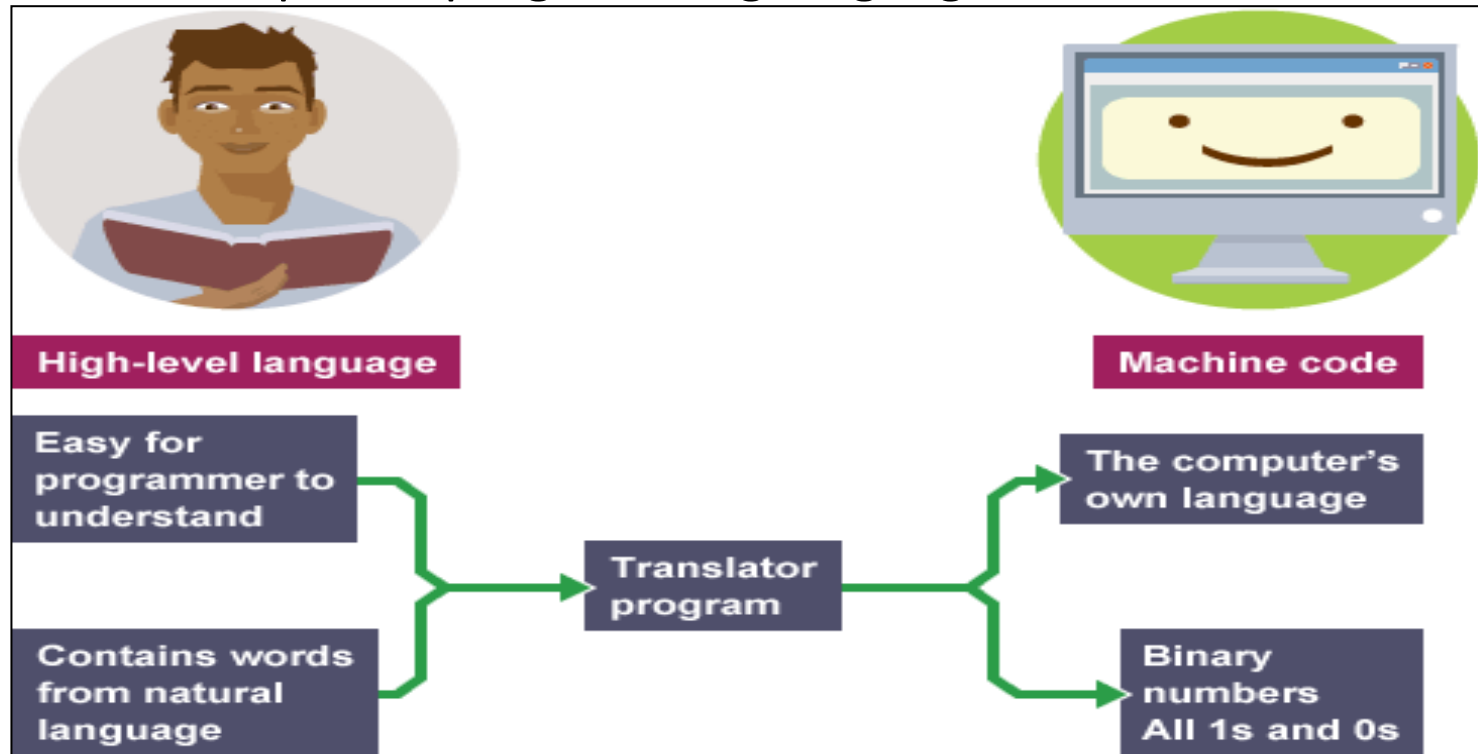
INTRODUCTION TO COMPILATION AND LEXICAL ANALYSIS



Dr. B.V. Baiju,
SCOPE, Assistant Professor
VIT, Vellore

INTRODUCTION TO PROGRAMMING LANGUAGE TRANSLATORS

- Language translators allow computer programmers to write *sets of instructions* in specific programming languages.



Different Programming Languages

```
01010010001111  
0000000110101  
01000110111101
```

**Machine
language**



```
mov sec ebx  
mov six e9c  
mov rax rdx
```

**Assembly
language**



```
#include <stdio.h>  
int main() {  
    printf ("Hello World");  
    return 0; }
```

**High-level
language**



C = A + B;

C

C++

JAVA

High Level Language

ADD A , B

Assembly Language

100100111

Machine Language

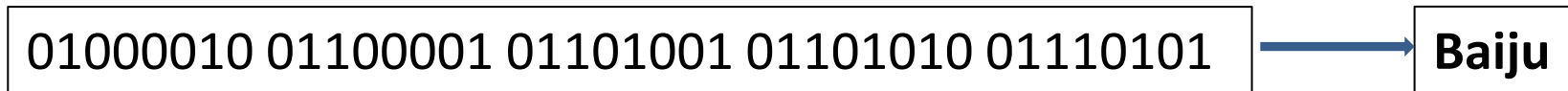


Hardware

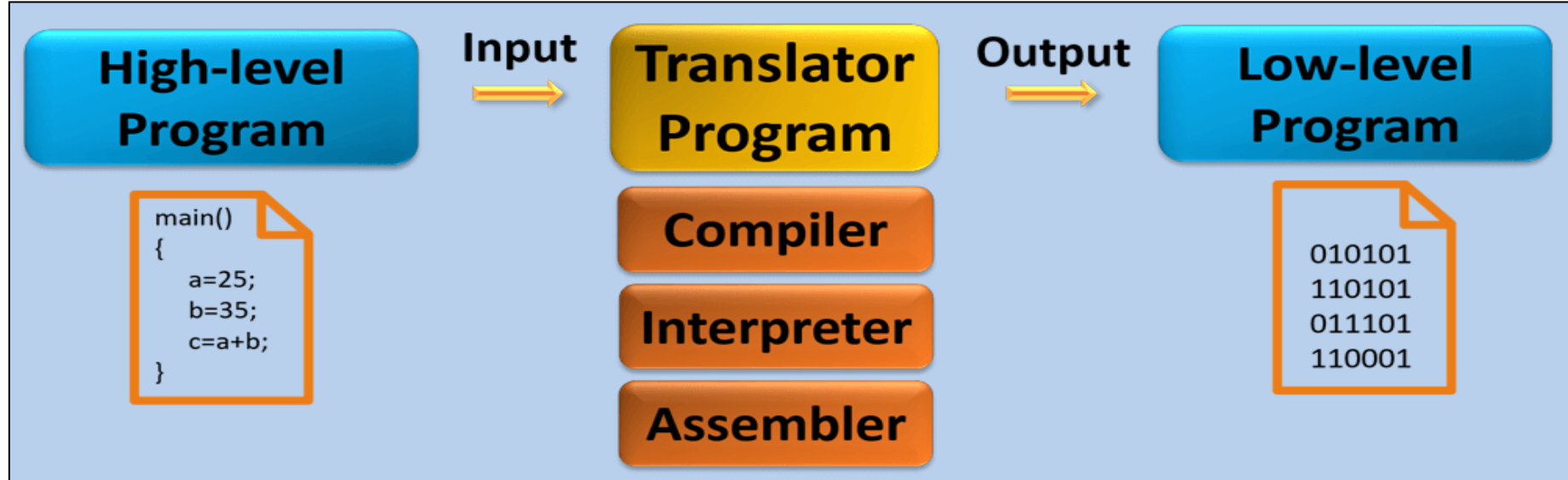
- A translator is a programming language processor that converts a computer program from one language to another.
- It takes a program written in source code and converts it into machine code.
- These instructions are converted by the language translator into machine code.
- It discovers and identifies the error during translation.



- Language translators are also called language processors.

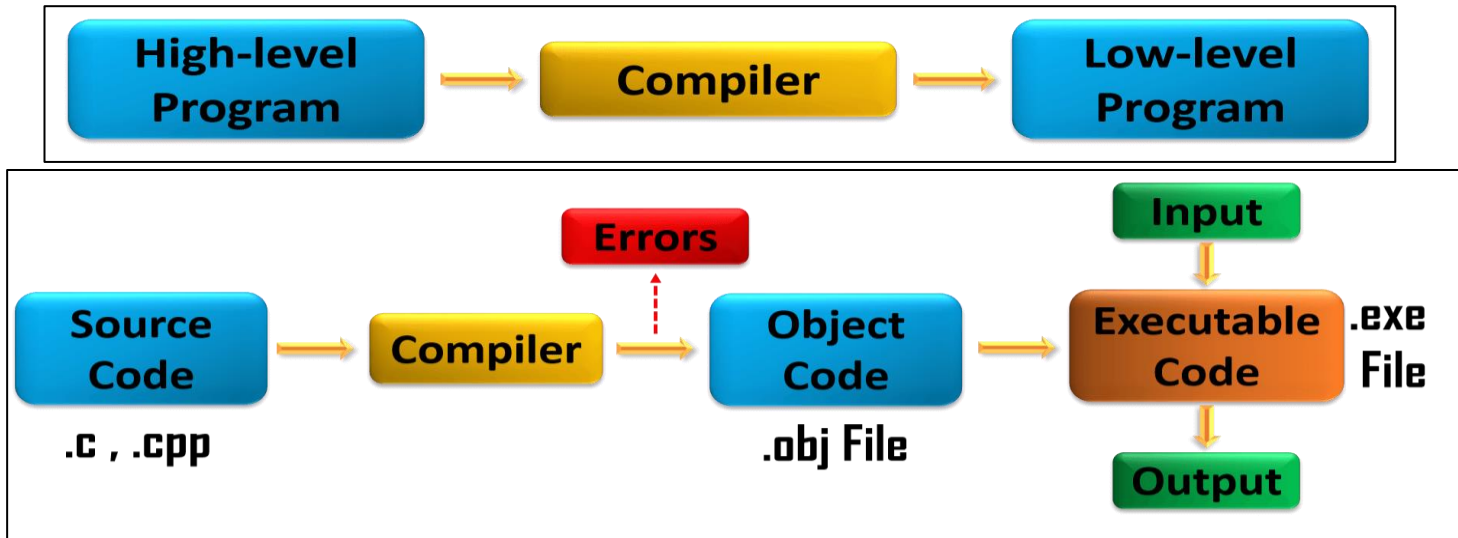


- There are mainly three types of translators that are used to translate different programming languages into machine-equivalent code:



1. COMPILER

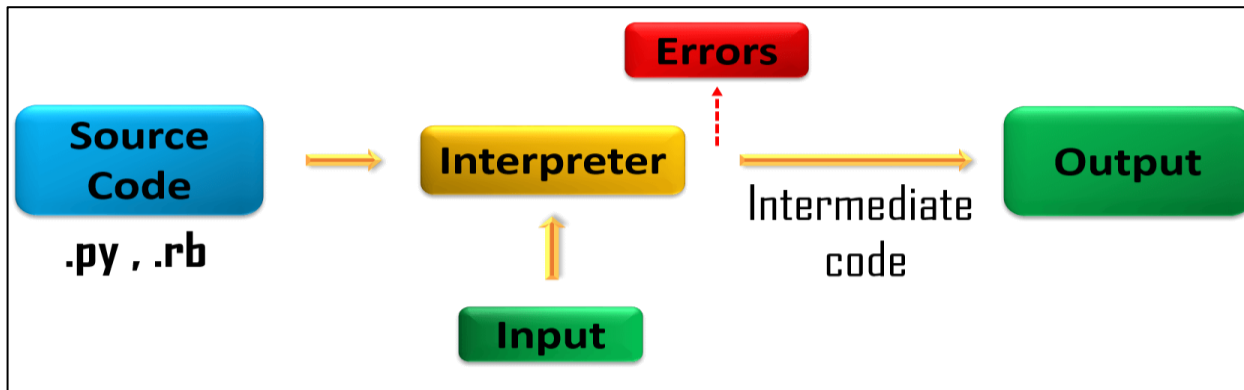
- A compiler is a translator used to *convert high-level programming language to low-level programming language*.
- It converts the whole program in one session and reports errors detected after the conversion.
- Languages that use compilers are C++, C, and Java.



- The compiler takes time to do its work as it translates high-level code to lower-level code all at once and then saves it to memory.

2. INTERPRETER

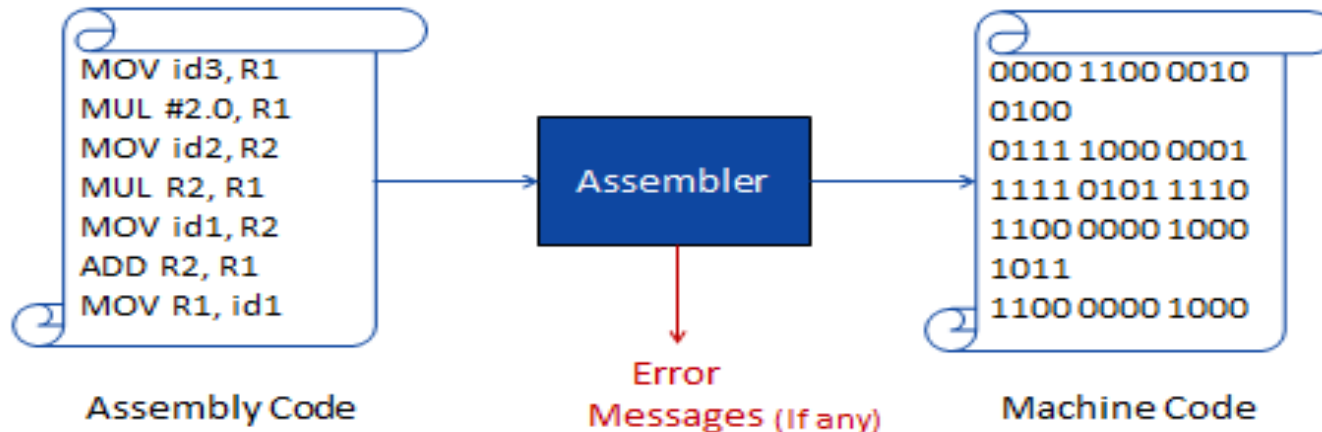
- An Interpreter is a translator that effectively accepts a source program and executes it directly *without producing any object code first*.
- Interpreter *translates line by line* and reports the error once it encountered during the translation process.
- An interpreter is *faster than a compiler* as it immediately executes the code upon reading the code.
- An interpreter *provides detailed error messages* and *supports interactive debugging*.



- Programming language like *Python, Ruby* use interpreters.

3. ASSEMBLER

- Assemblers translate a program written in assembly language into machine language.
- Assembly language is a low-level programming language.
- It translates by evaluating the symbols in the source code and generates the machine code accordingly.

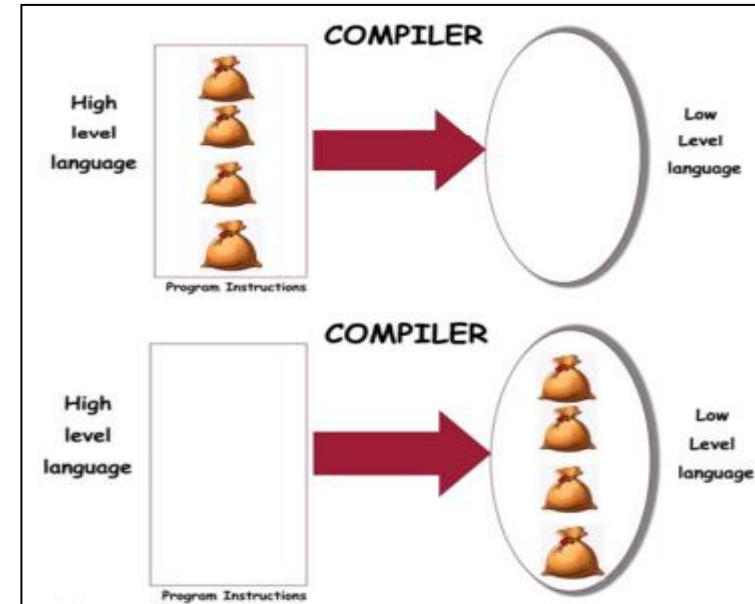
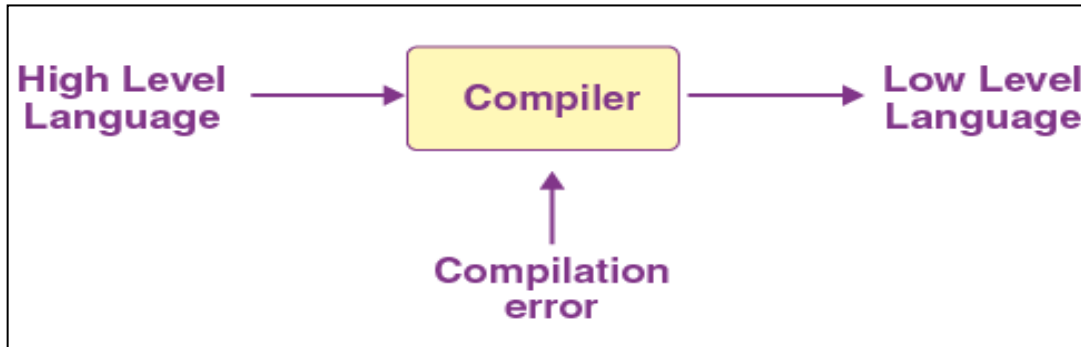


PARAMETERS	COMPILER	INTERPRETER	ASSEMBLER
Conversion	It converts the high-defined programming language into Machine language or binary code.	It also converts the program-developed code into machine language or binary code.	It converts programs written in the assembly language to the machine language or binary code.
Scanning	It scans the entire program before converting it into binary code.	It translates the program line by line to the equivalent machine code.	It converts the source code into the object code then converts it into the machine code.
Error Detection	Gives the full error report after the whole scan.	Detects error line by line. And stops scanning until the error in the previous line is solved.	It detects errors in the first phase, after fixation the second phase starts.
Code generation	Intermediate code generation is done in the case of Compiler.	There is no intermediate code generation.	There is an intermediate object code generation.
Execution time	It takes less execution time comparing to an interpreter.	An interpreter takes more execution time than the compiler.	It takes more time than the compiler.
Examples	C, C#, Java, C++	Python, Perl, VB, PostScript, LISP, etc...	GAS, GNU

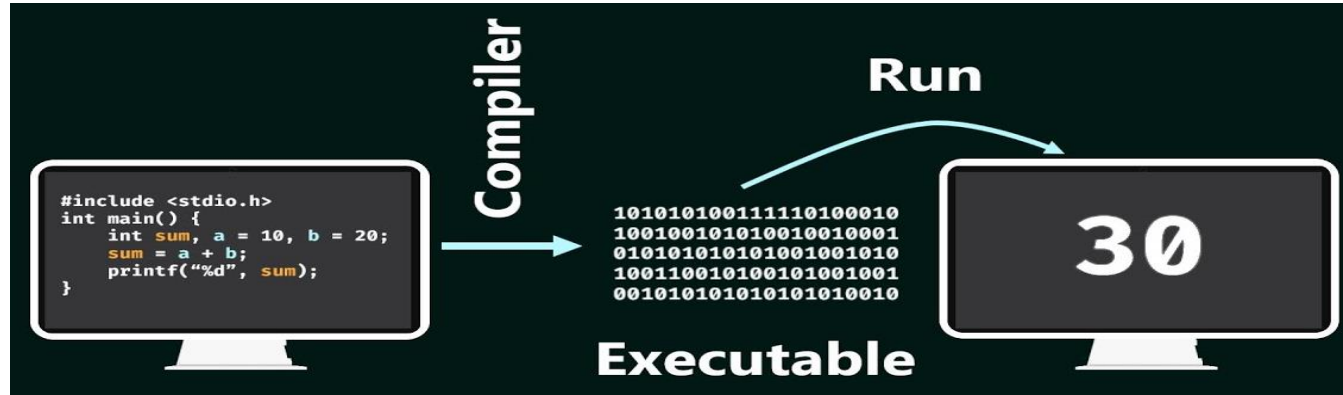
STRUCTURE AND PHASES OF A COMPILER

Compiler

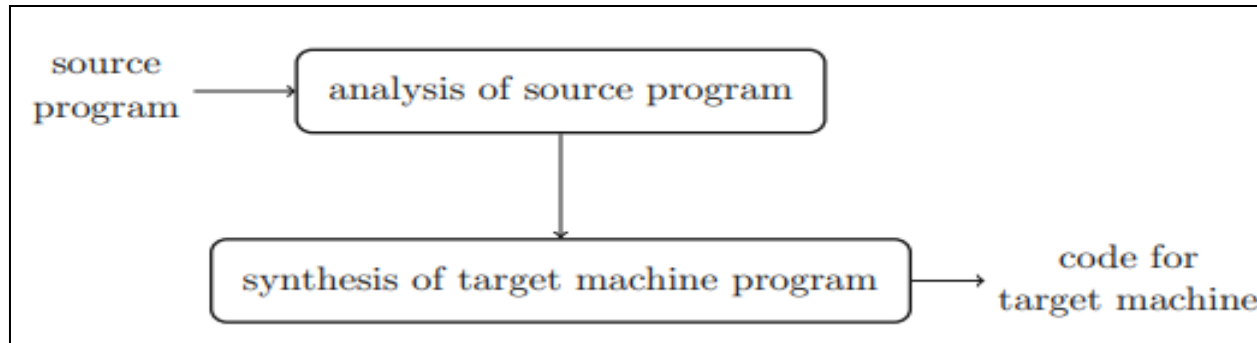
- A compiler is a translator that converts the **high-level language** into the **machine language**.
- High-level language is written by a developer and machine language can be understood by the processor.
- Compiler is used to **show errors** to the programmer.



- The main purpose of compiler is to change the code written in one language without changing the meaning of the program.

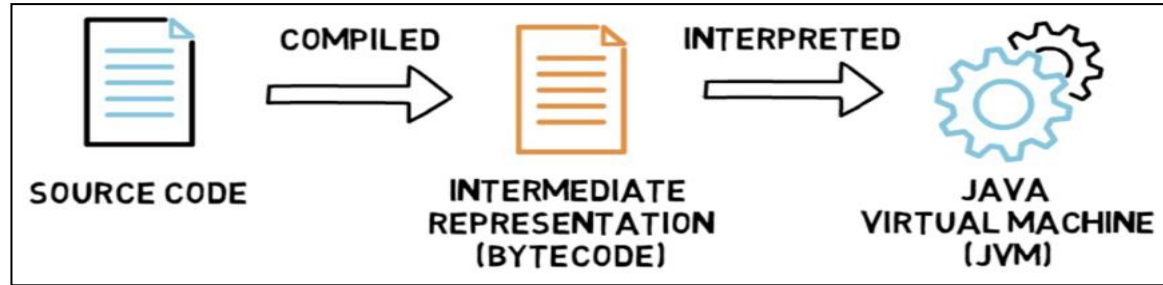


- The compilation is divided into two phases:
 - **Analysis** (Machine Independent/Language Dependent)
 - **Synthesis** (Machine Dependent/Language-Independent)



1. Analysis (Front end of a compiler)

- Analysis phase **reads** the source program and **splits** it into multiple tokens and constructs the **intermediate representation** of the source program.



- It also checks the program for some errors like **lexical errors**, **grammar errors**, and **syntax errors**.
- The analysis part also collects information about the source program and stores it in a data structure called a **symbol table**.
- Symbol table will be used all over the compilation process.

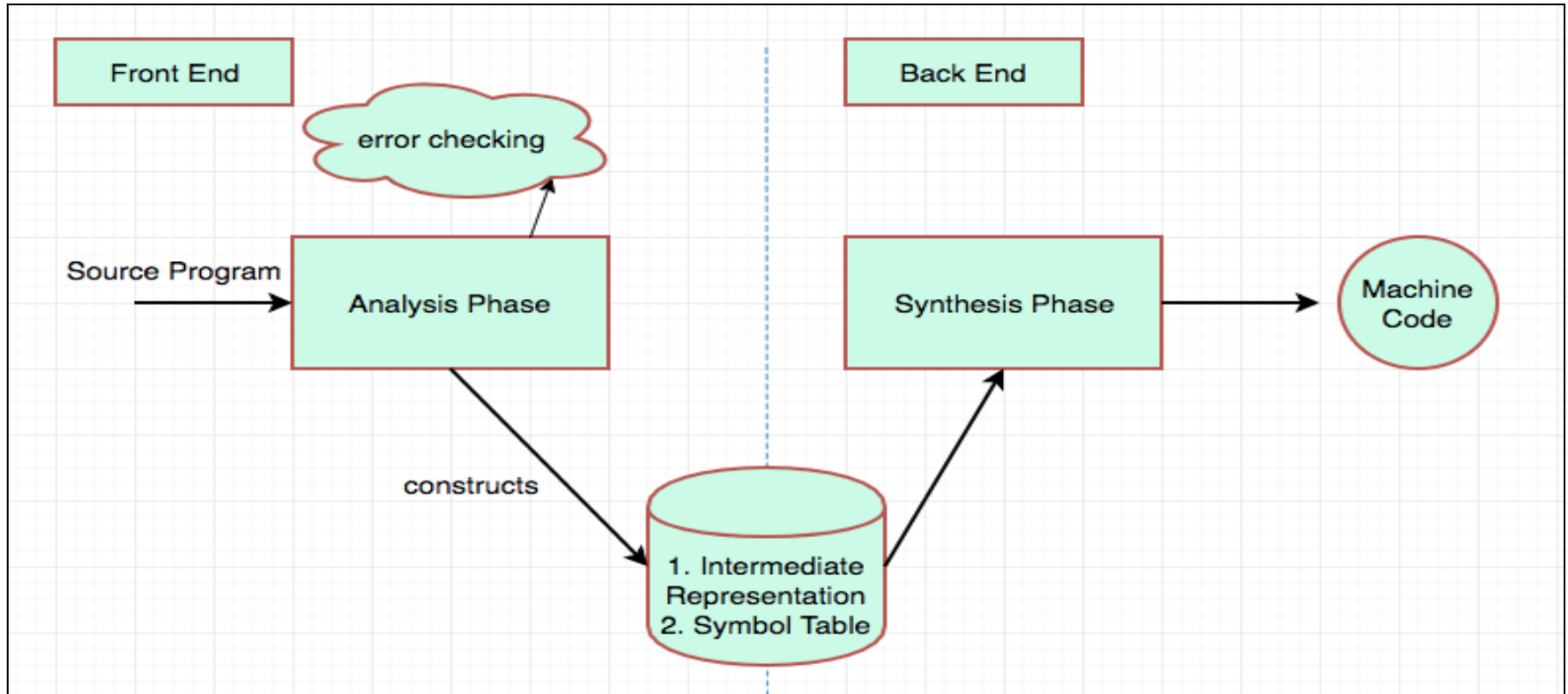
```
int count;  
char x[] = "NESO ACADEMY";
```

Entries

Name	Type	Size	Dimension	Line of Declaration	Line of Usage	Address
count	int	2	0	--	--	--
x	char	12	1	--	--	--

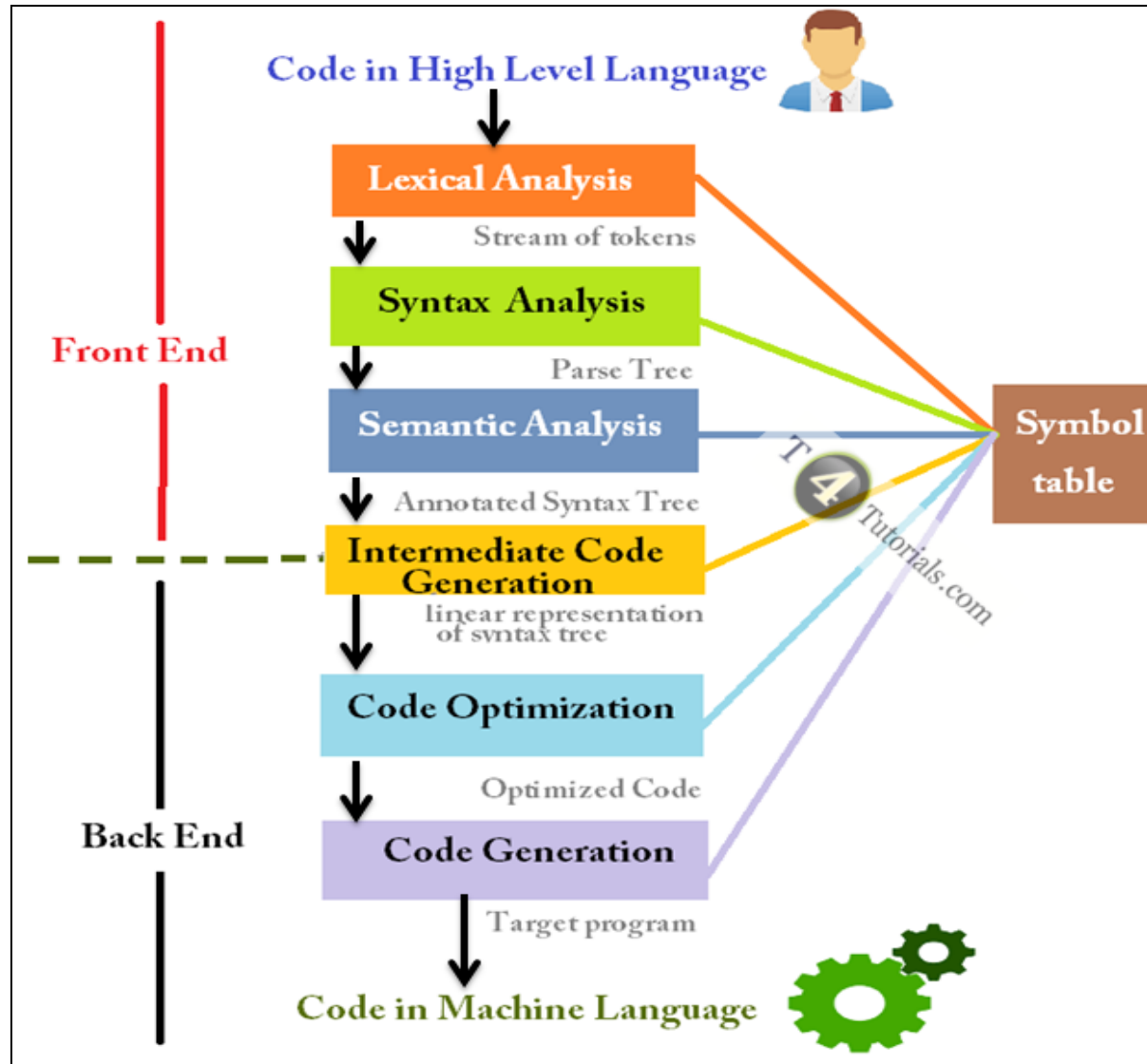
2. Synthesis (Back end of a compiler)

- It will get the analysis phase input(intermediate representation and symbol table) and produces the targeted **machine level code**.



Phases of compilation

- The compilation process contains the *sequence of various phases*.
- Each phase takes source program in **one representation** and produces output in **another representation**.
- *Each phase takes input from its previous stage.*
- The symbol table, which stores information about the entire source program, is used by all phases of the compiler.



`x = a+b*c;`



Compiler



```
⋮  
mov     eax, DWORD PTR [rbp-8]  
imul    eax, DWORD PTR [rbp-12]  
mov     edx, eax  
mov     eax, DWORD PTR [rbp-4]  
add     eax, edx  
mov     DWORD PTR [rbp-16], eax  
⋮
```

Lexical Analysis

Syntax Analysis

Semantic Analysis

Intermediate Code
Generation

Code Optimization

Target Code Generation

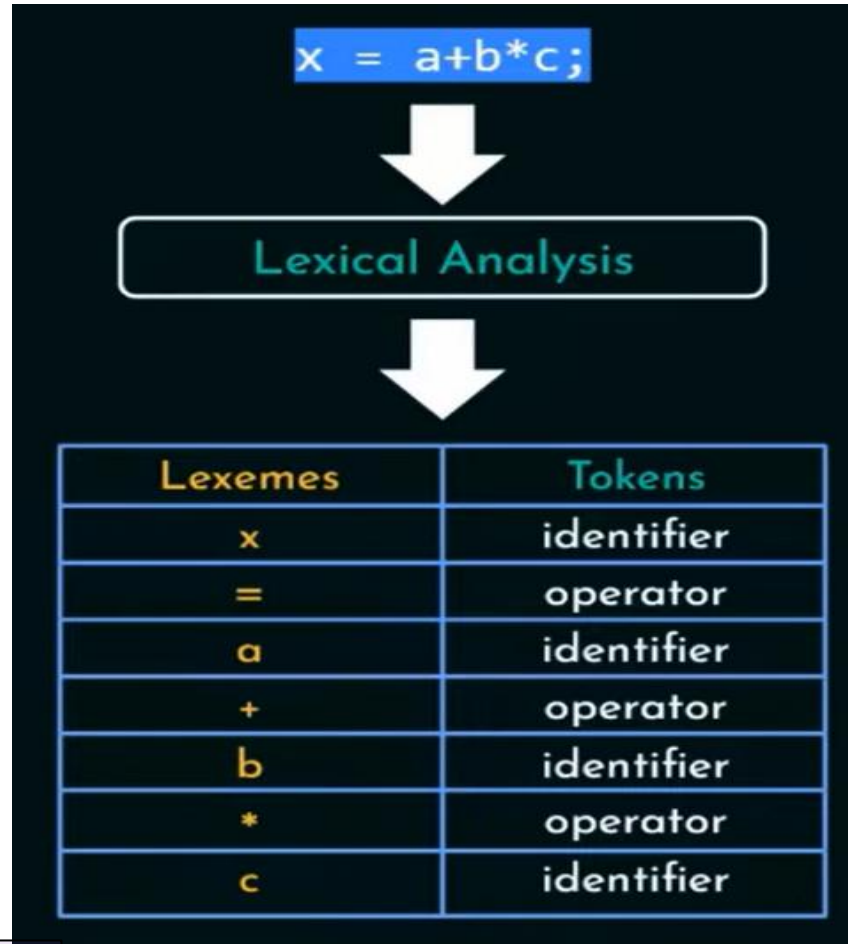
a. Lexical Analysis

- The first phase of a compiler is called lexical analysis or **scanning**.
- The lexical analyzer reads the **stream of characters** making up the source program and **groups the characters** into meaningful sequences called **lexemes**.
- This process can be **left to right** and **character by character**.
- The primary functions of this phase are:
 - Identify the **lexical units** in a source code
 - Classify lexical units into classes like **constants**, **reserved words**, and enter them in different tables.
 - It will **ignore comments** in the source program
 - Identify token which is not a part of the language

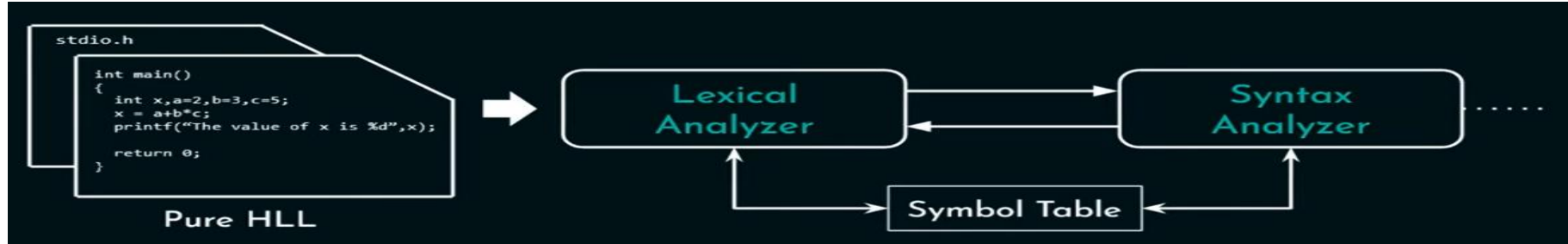


- For each lexeme, the lexical analyzer produces as output a *token* of the form

(token-name, attribute-value)



- The output of the lexical analysis is a sequence of tokens, which will be further sent to the syntax analysis as an input.



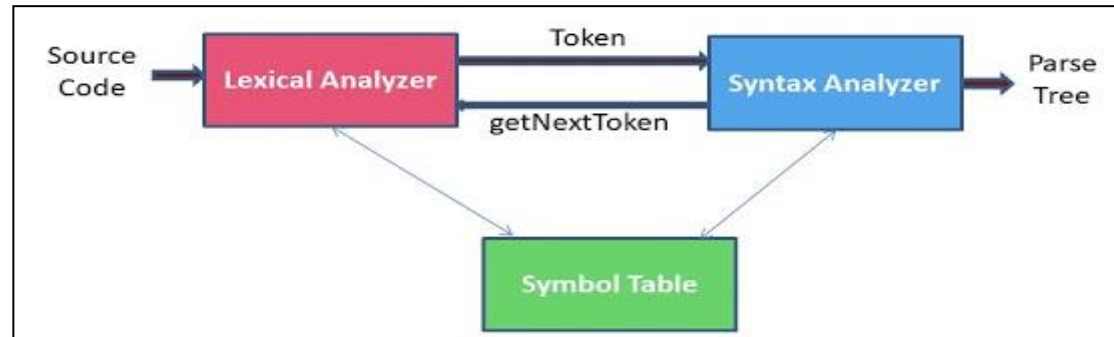
```
program gcd (input, output);
var i, j : integer;
begin
    read (i, j);
    while i <> j do
        if i > j then i := i - j else j := j - i;
    writeln (i)
end.
```

↓

```
program gcd ( input , output ) ;
var i , j : integer ; begin
read ( i , j ) ; while
i <> j do if i > j
then i := i - j else j
:= i - j ; writeln ( i
) end .
```

b. Syntax Analysis

- The second phase of the compiler is syntax analysis or **parsing**.
- It determines whether or not a **text follows the expected format**.
- The main aim of this phase is to make sure that the source code was written by the programmer is correct or not.
- Syntax analysis is based on the rules based on the specific programming language by **constructing the parse tree** with the help of tokens.
- List of tasks performed in this phase
 - **Obtain tokens from the lexical analyzer.**
 - **Checks if the expression is syntactically correct or not.**
 - **Report all syntax errors.**
 - **Construct a hierarchical structure which is known as a parse tree.**



- Example

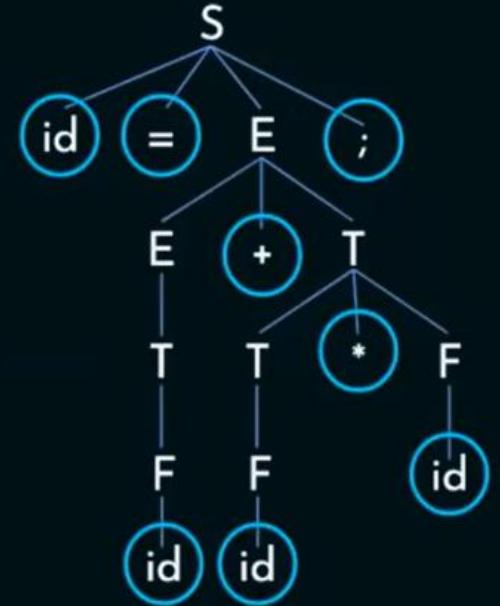
Lexemes	Tokens
x	identifier
=	operator
a	identifier
+	operator
b	identifier
*	operator
c	identifier

x = a+b*c;

Syntax Analysis

$S \rightarrow id = E ;$
 $E \rightarrow E + T | T$
 $T \rightarrow T * F | F$
 $F \rightarrow id$

id = id + id * id ;



Parse Tree

c. Semantic Analysis

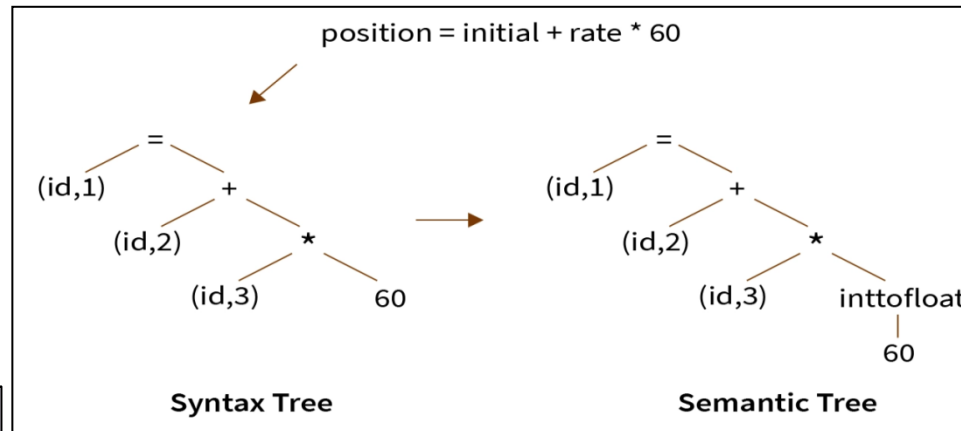
- Semantic analysis checks the **semantic consistency** of the code.
- It uses the **syntax tree** of the previous phase along with the **symbol table** to verify that the given source code is semantically consistent.
- It also checks whether the code is conveying an **appropriate meaning**.
- An important part of semantic analysis is **type checking**, where the compiler checks that each operator has matching operands.
- Functions of Semantic analyses phase are:
 - *Helps you to **store type information** gathered and save it in symbol table or syntax tree.*
 - *Allows you to perform type checking.*
 - *In the case of type mismatch, where there are no exact type correction rules which satisfy the desired operation a semantic error is shown.*
 - *Collects type information and checks for type compatibility.*
 - *Checks if the source language permits the operands or not.*

- **Static semantic checks (done by the compiler) are performed at compile time**
 - *Type checking*
 - *Every variable is declared before used*
 - *Identifiers are used in appropriate contexts*
 - *Check subroutine call arguments*
 - *Check labels*
- **Dynamic semantic checks are performed at run time, and the compiler produces code that performs these checks**
 - *Array subscript values are within bounds*
 - *Arithmetic errors, e.g. division by zero*
 - *Pointers are not dereferenced unless pointing to valid object*
 - *A variable is used but hasn't been initialized*
 - *When a check fails at run time, an exception is raised*

Example

```
float x = 20.2;  
float y = x*30;
```

The semantic analyzer will typecast the integer 30 to float 30.0 before multiplication



d. Intermediate Code Generation

- In the intermediate code generation, compiler generates the **source code into the intermediate code**.
- Intermediate code is generated between the **high-level language** and the **machine language**.
- The intermediate code should be generated in such a way that you can easily translate it into the target machine code.
- The two most important kinds of intermediate representations are:
 - **Tree** i.e. “parse trees” and “syntax tree”.
 - **A Linear representation** i.e., “three address code”.

Example

total = count + rate * 5

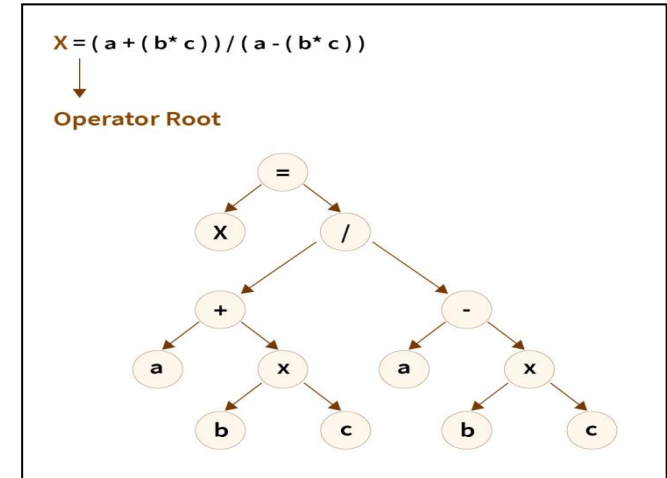
Intermediate code with the help of address code method is:

t1 := int_to_float(5)

t2 := rate * t1

t3 := count + t2

total := t3



Semantically
Verified
Parse Tree



Intermediate Code
Generation



Intermediate
Code



$id = id + id * id ;$



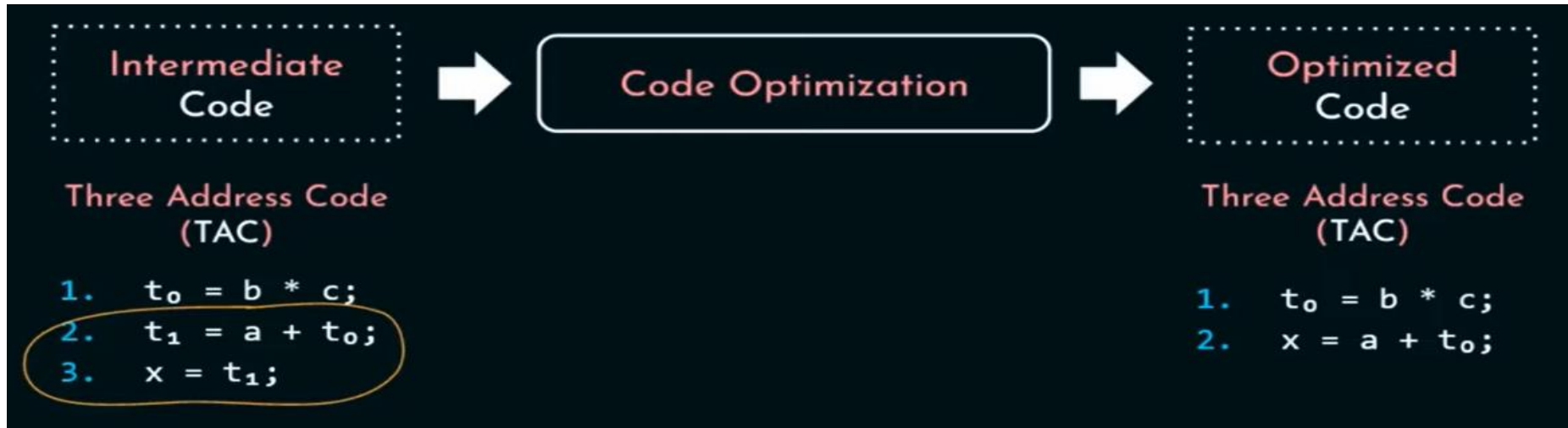
$x = a + b * c ;$

Three Address Code
(TAC)

1. $t_0 = b * c ;$
2. $t_1 = a + t_0 ;$
3. $x = t_1 ;$

e. Code Optimization

- Code optimization is an optional phase.
- It is used to **improve the intermediate code** so that the output of the program could run faster and take less space.
- It **removes the unnecessary lines of the code** and arranges the sequence of statements in order to speed up the program execution.
- **Removing unreachable code** and getting rid of **unused variables**
- Removing statements which are not altered from the loop



f. Code Generation

- Code generation is the last and final phase of a compiler.
- It gets inputs from code optimization phases and produces the **page code** or **object code** as a result.
- Objective is to **allocate storage** and **generate relocatable machine code**.
- It also **allocates memory locations** for the **variable**.
- The instructions in the intermediate code are converted into machine instructions.
- This phase converts the optimize or intermediate code into the **target language** (Machine Code)



Position := initial + rate * 60

Lexical analyser

id1 := id2 + id3 * 60

Syntax analyser

```
graph TD
    A["id1 := id2 + id3 * 60"] --> B["id1"]
    A --> C["+"]
    C --> D[id2]
    C --> E["*"]
    E --> F[id3]
    E --> G[60]
```

Semantic analyser

Semantic analyser

```
graph TD
    A["id1 := id2 + id3 * 60"] --> B[id1]
    A --> C["+"]
    C --> D[id2]
    C --> E["*"]
    E --> F[id3]
    E --> G[Inttoreal]
    G --> H[60]
```

Intermediate code generation

```
temp1 := inttoreal (60)
temp2 := id3 * temp1
temp3 := id2 * temp2
id1 := temp3
```

Code optimization

```
temp1 := id3 * 60.0
id1 := id2 + temp1
```

Code generation

```
MOVf id3, R2
MULF #60.0, R2
MOVf id2, R1
ADDF R2, R1
MOVf R1, id1
```

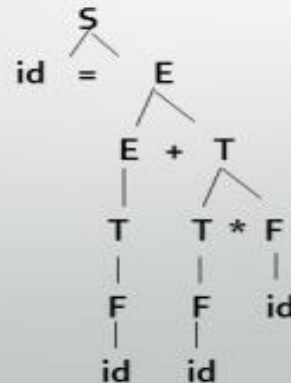
EXAMPLE

$a = x + y * z;$ (source program)

LEXICAL ANALYSER

$id = id + id * id$

SYNTAX ANALYSER



Grammar

$S \rightarrow id = E;$
 $E \rightarrow E + T / T$
 $T \rightarrow T * F / F$
 $F \rightarrow id$

S – Symbol
Id – identifier
E - Expression
T- Term
F - Factor

SEMANTIC ANALYSER

Semantically Verified
Parse Tree

INTERMEDIATE CODE GENERATION

$t1 = y * z$
 $t2 = x + t1$
 $a = t2$

CODE OPTIMIZATION

$t1 = y * z$
 $a = x + t1$

TARGET CODE GENERATION

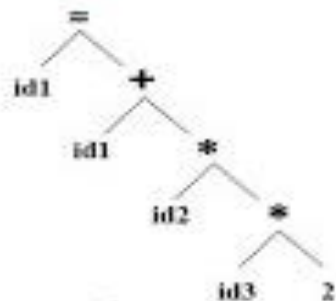
Mul	R1	R2		$x \rightarrow R0$
Add	R0	R2		$y \rightarrow R1$
Mov	R2	a		$z \rightarrow R2$

$a = a + b * c * 2$

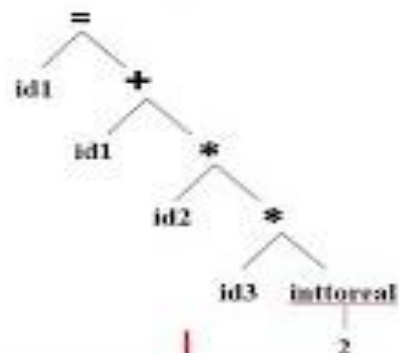
Lexical analysis

$id1 = id1 + id2 * id3 * 2$

Syntax analysis



Semantic analysis



Intermediate code generation

$t1 = \text{inttoreal}(2)$
 $t2 = id3 * t1$
 $t3 = id2 * t2$
 $t4 = id1 + t3$
 $id1 = t4$

Code Optimization

As per question, not need to optimize intermediate code

Code Generation

MOV R1, #2
MOV R2, id3
MUL R2, R1
MOV R3, id2
MUL R3, R2
MOV R4, id1
ADD R4, R3
MOV id1, R4

Use of symbol tables in different phases

